Nome: Vinicius Ramon Ferreira **RA:** 24360978-2

Atividade - Aula 013

Exercício em duplas: o que o Prolog responde? (1/3)

Para cada item: **(a)** o que o Prolog responde? **(b)** qual regra explica?

1 · Unificação

```
?- f(X, b) = f(a, Y).

?- f(X, X) = f(a, b).
```

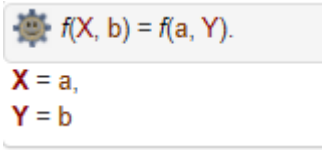
2 · Aritmética

```
?- X = 2 + 3.

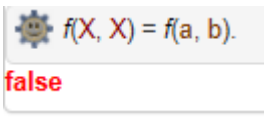
?- X is 2 + 3.

?- 5 = 2 + 3.
```

1 - Unificação

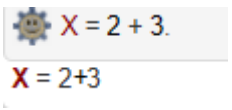


?- f(X, b) = f(a, Y).
(a) X = a, Y = b.
(b) O Prolog unifica os argumentos um a um: X com a, e b com Y. Como as duas estruturas têm o mesmo nome e a mesma aridade e as variáveis estão livres, a unificação dá certo.

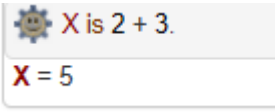


?- f(X, X) = f(a, b).
(a) false.
(b) O primeiro X é unificado com a, e então o segundo X (que já vale a) precisa unificar com b. Como a e b são átomos diferentes, falha. A mesma variável não pode ter dois valores diferentes.

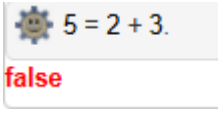
2 - Aritmética



?- X = 2 + 3.
(a) X = 2+3.
(b) O = é unificação, não faz conta. 2 + 3 é só uma estrutura +(2, 3), e X passa a ser essa estrutura.



?- X is 2 + 3.
(a) X = 5.
(b) O is avalia a expressão do lado direito e unifica o resultado com o lado esquerdo.



?- 5 = 2 + 3.
(a) false.
(b) Como o = só unifica, o número 5 é comparado com a estrutura +(2, 3), e os dois são termos diferentes, então não unificam. Para comparar o valor seria 5 =:= 2 + 3 (que dá true).

Exercício: o que o Prolog responde? (2/3)

Para cada item: **(a)** o que o Prolog responde? **(b)** qual regra explica?

3 · Ordem das respostas

```
cor(maca, vermelha).
cor(banana, amarela).
cor(morango, vermelha).

?- cor(F, vermelha).
```

4 · Listas

```
?- [H | T] = [a, b, c].

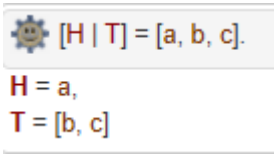
?- [A, B | R] = [1, 2].
```

3 - Ordem das respostas

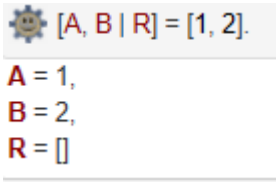


?- cor(F, vermelha).
(a) F = maca ; F = morango. (primeiro aparece F = maca, e ao pedir mais respostas com ";", aparece F = morango)
(b) O Prolog procura na base de cima para baixo, na ordem em que os fatos foram escritos. Acha maca primeiro, e ao pedir outra resposta ele faz backtracking e continua procurando, pulando a banana (amarela) e achando morango.

4 - Listas



?- [H | T] = [a, b, c].
(a) H = a, T = [b, c].
(b) O padrão [H | T] separa a cabeça (primeiro elemento) da cauda (resto da lista).



?- [A, B | R] = [1, 2].
(a) A = 1, B = 2, R = [].
(b) O padrão pega os dois primeiros elementos, e o que sobra é a lista vazia, pois a lista só tinha dois elementos. A cauda de uma lista sempre é uma lista, mesmo vazia.

Exercício: o que o Prolog responde? (3/3)

Para cada item: **(a)** o que o Prolog responde? **(b)** qual regra explica?

5 · Negação

```
gosta(ana, prolog).

?- \+ gosta(bruno, prolog).

?- \+ gosta(X, prolog).
```

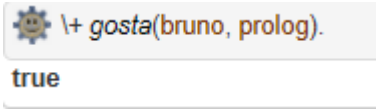
6 · Ordem das cláusulas

```
ancestral(X, Y) :-
    ancestral(Z, Y), pai(X, Z).
ancestral(X, Y) :- pai(X, Y).
pai(ana, bia).

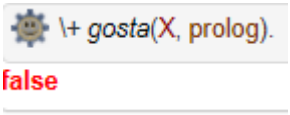
?- ancestral(ana, bia).
```

5 - Negação

gosta(ana, prolog).



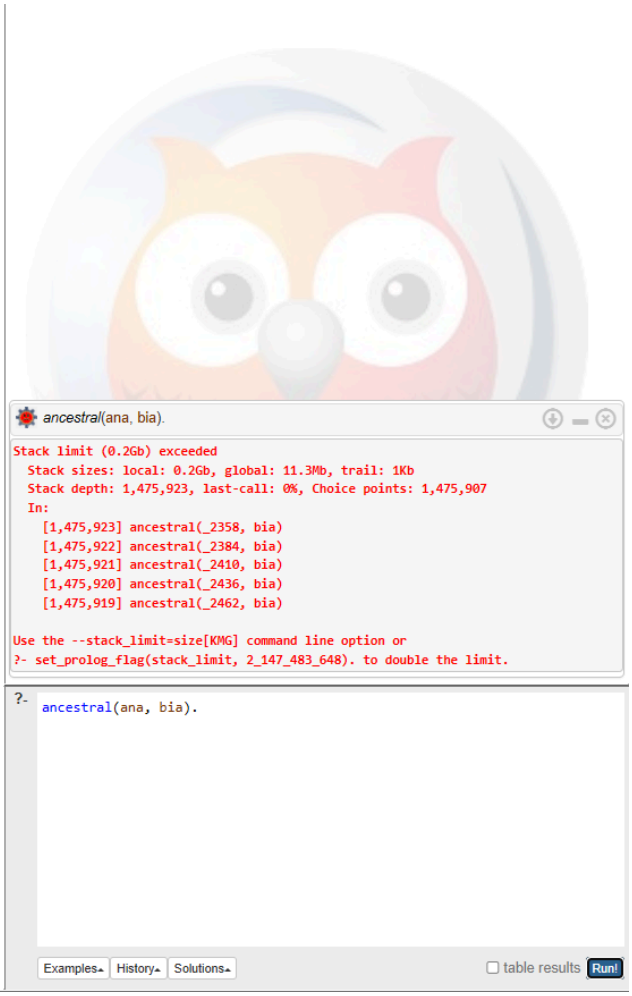
?- \+ gosta(bruno, prolog).
(a) true.
(b) O \+ é negação por falha: o Prolog tenta provar gosta(bruno, prolog), não consegue (não está na base), e então o \+ tem sucesso. Ele assume que o que não pode ser provado é falso (mundo fechado).



?- \+ gosta(X, prolog).
(a) false.
(b) gosta(X, prolog) consegue ser provado com X = ana, então o \+ falha. Isso não quer dizer "alguém não gosta", e sim que existe pelo menos um que gosta. Além disso, o \+ nunca devolve valor para X.

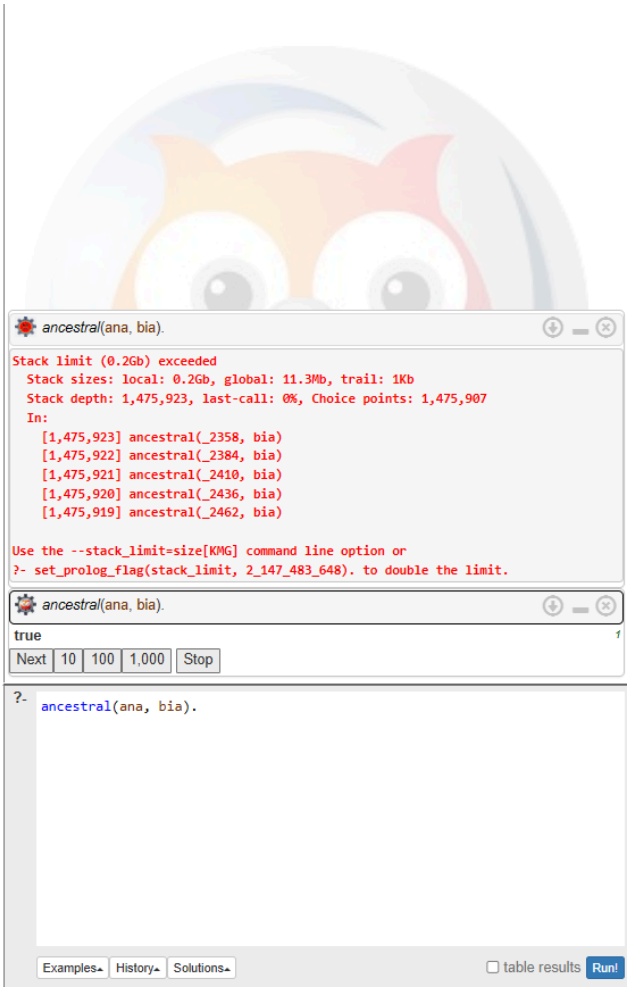
6 - Ordem das cláusulas

```
ancestral(X, Y) :-
    ancestral(Z, Y), pai(X, Z).
ancestral(X, Y) :- pai(X, Y).
pai(ana, bia).
```



?- ancestral(ana, bia).
(a) Não responde: entra em recursão infinita e termina com erro de estouro de pilha (algo como "Stack limit (1.0Gb) exceeded").
(b) O Prolog usa busca em profundidade, testando as cláusulas de cima para baixo e os objetivos da esquerda para a direita. A primeira cláusula de ancestral começa chamando ancestral(Z, Y), que de novo usa a primeira cláusula, e assim por diante, sem nunca chegar no pai(X, Z) nem na cláusula base. É a recursão à esquerda. Corrigindo a ordem, com a cláusula base primeiro e o pai antes da chamada recursiva, funciona:

```
ancestral(X, Y) :- pai(X, Y).
ancestral(X, Y) :- pai(X, Z), ancestral(Z, Y).
pai(ana, bia).
```



ancestral(X, Y) :- pai(X, Y).
ancestral(X, Y) :- pai(X, Z), ancestral(Z, Y).